

Murphy: Performance Diagnosis of Distributed Cloud Applications

Vipul Harsh, Wenxuan Zhou, Sachin Ashok, Radhika Niranjan Mysore, P.
Brighten Godfrey, Sujata Banerjee

Affiliations: University of Illinois Urbana-Champaign, VMware, Inc.

ACM SIGCOMM '23

Goal

- Automatically diagnose performance problems in distributed cloud applications
- Given an observed symptom, identify the entities most likely responsible for causing it
- Work using commonly available monitoring telemetry, rather than requiring fine-grained causal traces or a clean DAG structure
- Produce results that are accurate and actionable for operators, ideally with a ranked set of likely causes and an explanation chain

Motivation

- Diagnosing performance incidents in modern cloud applications is hard because systems contain many interacting application and infrastructure components
- Operators already know symptom(s), but they cannot know which component caused it
- Previous methods have weaknesses such as ignoring dependencies, requiring DAG-structured dependency data, or requiring fine-grained causal traces
- Want to formulate diagnosis method that works with commonly available monitoring telemetry and still reasons accurately in messy real-world settings

Enterprise Networks

- Target application is medium-to-large enterprise cloud environments
- Engineers use monitoring tools to gain insight into network
- May provide valuable information like entity metadata and time-series performance metrics
- Environment is vast and subject to changes, so diagnosing a problem becomes difficult

Entity type		Example metrics
VM		CPU utilization, memory utilization, network transmit rate, receive rate, packet drops, disk read/write rate
Host		<i>Metrics similar to VM metrics</i>
Container		<i>Metrics similar to VM metrics</i>
Virtual NIC		Transmit rate, receive rate, dropped packets
Physical NIC		Transmit rate, receive rate, dropped packets, latency, interface peak buffer utilization
Flow		Session count, throughput, RTT, packet loss, retransmission ratio
Switch	inter-face	Network rate, dropped packets, latency, interface peak buffer utilization
Datastore		Space utilization

Need for Cyclic Dependencies

- May have cyclic relationships in system, such as if two VMs relied on same host
- Monitoring data does not reveal clear dependency at all
- Consider TCP flow between web front end VM to another VM where performance of application is not directly affected by this flow
- Expensive to acquire fine-grained time-series data to possibly determine primary direction of importance

Related Work

- Fault Localization: Finds hidden faulty components, like failed links, packet drops, etc.
- ExplainIT: Correlation-based root-cause analysis for time-series data, may provide entities that are associated with the symptom but not actual cause
- NetMedic: Uses DAG dependency graph based on pairwise correlation on historical values
- Sage: Uses probability distribution to identify which metrics for resources in microservice cause QoS violations, assumes DAG structure
- Provenance-Based Methods: Consider all events and causality between them to find root cause of error in DAG structure

Murphy

- Murphy is a root-cause diagnosis framework that uses only common monitoring telemetry and a cyclic relationship graph to rank the entities most likely responsible for an observed performance symptom
- Constructs relationship graph using network data
- Given symptom, generate candidate root causes using Markov random field (MRF)
- Generate explanations for symptoms using path starting from predicted root cause

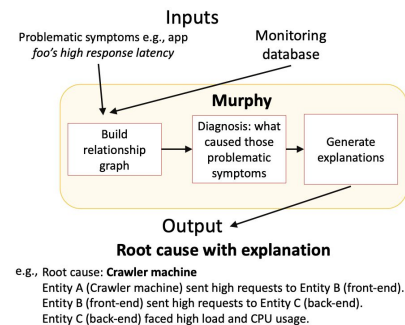


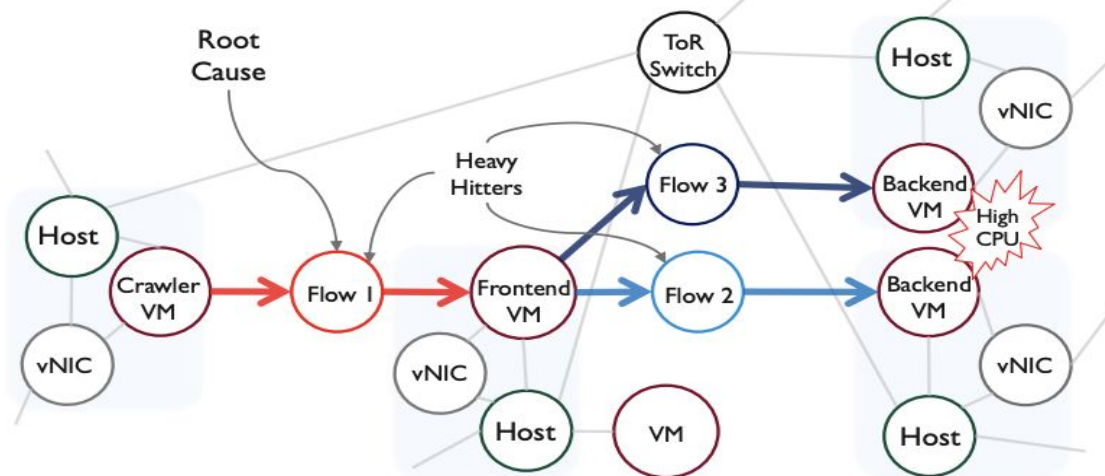
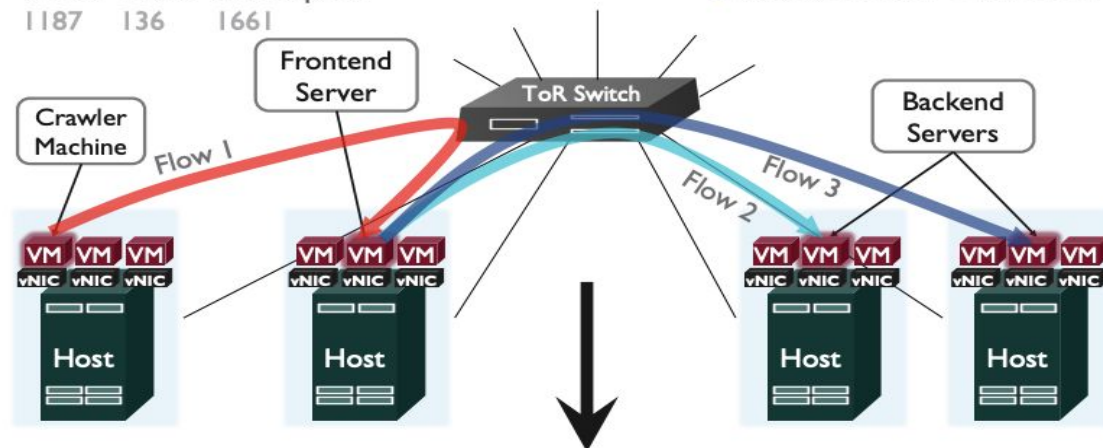
Figure 2: Murphy workflow

Relationship Graph

- Build a relationship graph whose edges come from simple monitored neighborhood relations between entities
- Start from the relevant entity set S : either the affected application's entities or the single problematic entity
- Recursively expand the graph by exploring neighbors
- Stop after few hops if the graph becomes too large
- By default, add edges between neighbors in both directions to keep possible dependencies
- Include time-series metrics of entities in graph structure for inference

Flows	VMs	ToR switches
2502	3640	12
vNICs	Hosts	Switch ports
1187	136	1661

- Flow 1: Crawler → Frontend
- Flow 2: Frontend → Backend 1
- Flow 3: Frontend → Backend 2



Markov Random Field (MRF) Model

- Use a Markov random field (MRF) to model the joint distribution of metrics across all entities in the relationship graph
- Formulation for global distribution:

$$P_G = \frac{1}{Z} \prod_{v:V(G)} P_v(v|\text{in_nbrs}(v)).$$

Markov Random Field (MRF) Model (cont.)

- Murphy learns multivariate probability distribution using a standard model such as linear regression, GMM, neural networks, etc. on one metric over all other incoming neighboring entities' metrics, over all historical time steps
- P_v is learned per time slice

Training Details

- For each entity v , learn a local probabilistic model P_v from historical metric values by relating v 's metrics to the metrics of its incoming neighbors within each time slice
- Murphy does not use pre-trained model as base, trains online on fresh data
- For paper's evaluation, each P_v is trained using one week of data prior to the incident, giving a few hundred time points
- To reduce overfitting, Murphy keeps only the top $B=10$ neighbor metrics, based on their correlation with the metrics of v

Sampling Algorithm (Overview)

- For a problematic symptom at entity D, Murphy asks whether another entity A could be causing it
- Conducts counterfactual test, asks If A had better performance than it has now, if this would cause D to improve
- Murphy uses the learned graph model to simulate how a change at A would affect other entities and eventually D
- Since A and D may be far apart, Murphy must also update intermediate entities along the way.

Sampling Algorithm (cont.)

- Murphy changes entity A to a better hypothetical value called A'
- Finds the shortest-path subgraph from A to the problematic entity D
- We then resamples the entities on that path, starting near A and moving toward D
- This produces a new possible value for D under the intervention on A
- Murphy repeats this process for $W = 4$ rounds so effects can move through cycles

Sampling Algorithm (cont.)

- Murphy compares D value using the new value A prime and the one using the true current value of A
- It generates many samples for both cases and looks at the resulting values of D
- If D looks significantly better when Murphy uses A prime, then A is labeled a root cause
- To save time, Murphy first prunes the search and only tests suspicious nearby entities and confirmed root causes are then ranked by how anomalous their metrics are

Sampling Algorithm (cont.)

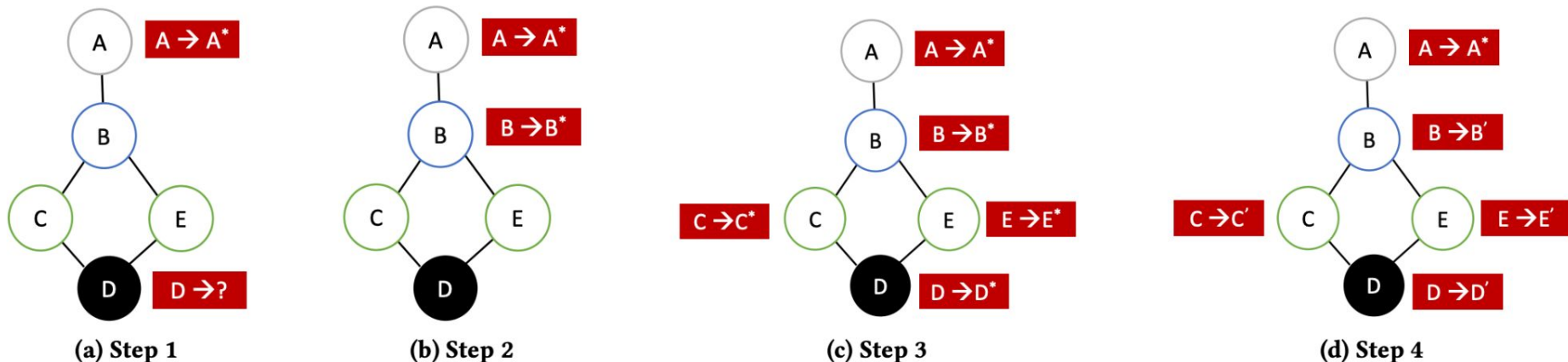


Figure 3: Inference algorithm- evaluating whether A is the root cause for D 's high metric value for a toy relationship graph with only one metric per entity. For clarity, we draw an undirected edge to represent a directed edge in both directions. We (a) change A to a counter-factual value A^* , (b) resample B assuming the new value A^* , (c) resample C, D and E in a similar way, and (d) resample C, D and E , again g times for Gibbs sampling. We run the same resampling algorithm, this time starting with the current true value of A , instead of a counterfactual, and obtained the resampled value D'' . After this sampling process, if $D' \ll D''$, then we classify A as a root cause for the high value of D .

Diagnosis

- After root causes are identified, Murphy generates human-readable explanations for them
- Each entity is assigned a label based on its current metrics, such as Faulty/Nonfunctional, Degraded performance, High drop rate, Heavy hitter, or Okay
- Reasons about holistic rules between entities' states

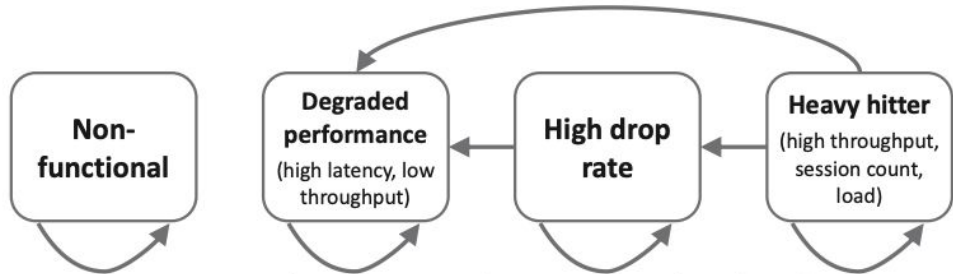
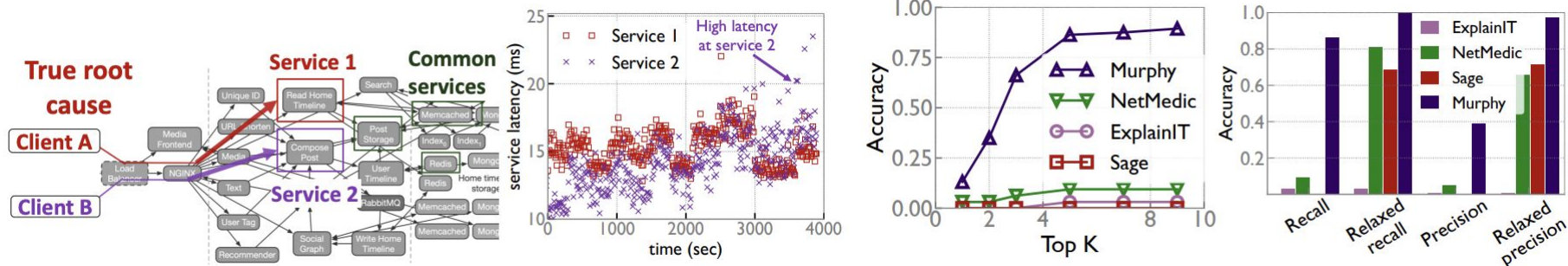


Figure 4: State machine encoding causal rules between entities' states for generating labeled explanation chains

Evaluation Setup

- Uses two environments: a large enterprise production environment and DeathStarBench microservice applications deployed on private servers and AWS
- For production environment, Use incident dataset with 13 real incidents and metrics dataset from over 300 production applications and 17,000 entities
- Microservice environment: generate traces from DeathStarBench and use controlled failure scenarios

Evaluation Results (Microservices)



(a) Cause: Performance interference

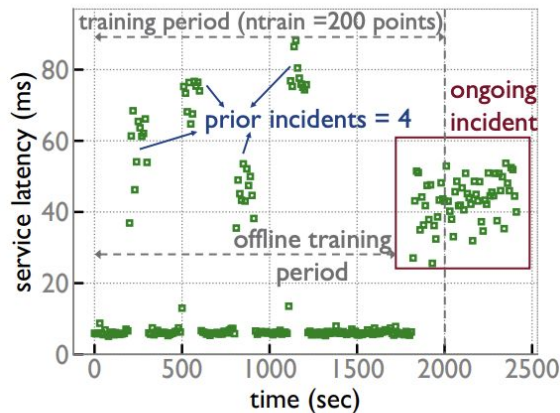
(b) Symptom: Latency spike

(c) Top-K Accuracy

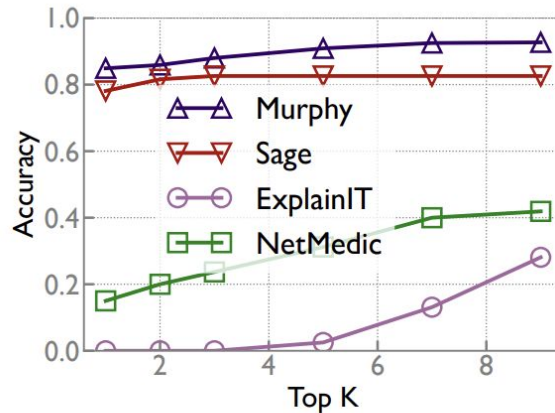
(d) Precision and recall

Figure 5: Performance interference experiment in DeathStarBench (§ 6.1). (a) Fault scenario: client A sends a lot of requests to service 1 which overwhelms the downstream common services shared between service 1 and 2, causing high latency for service 2 for Client B. “Common containers” are containers that “common services” reside on. (b) The fault gets injected just after 3000 seconds when Client A begins to send a lot of requests. (c) Accuracy (recall) in top-K. (d) Precision and recall (see § 6.1 for definition) of various schemes in producing the true root cause. Also shown is a relaxed notion of accuracy which measures how often a scheme gets at least one entity in top 5 that’s either the true root cause or a common container or a common service.

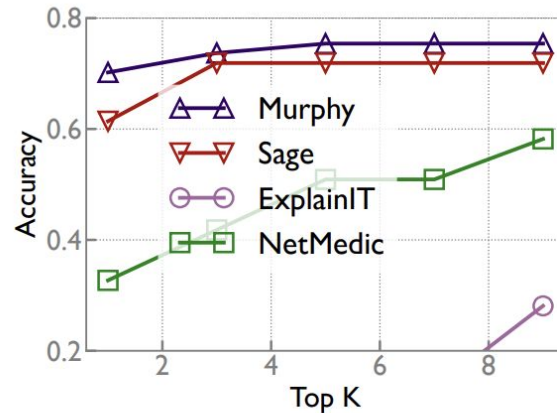
Evaluation Results (Microservices)



(a) A sample latency trace



(b) Top-K Acc (social-network)



(c) Top-K Acc (hotel-reserve)

Figure 6: Resource contention experiment on DeathStarBench applications: (a) service latency in a sample trace across time. The training period contains 4 prior incidents while the main incident is triggered at $t = 1800$ s. (b), (c) Top-K Accuracy (Acc) for varying K.

Evaluation Results (Production Environment)

Incident (observed problems)	Murphy	NetMedic	ExplainIT
	FPs	FPs	FPs
1. Two apps nodes crashed due to a plugin	6	69	93
2. App returning a 502 error	0	1	0
3. App unavailable	4	40	60
4. App slow, experiencing timeouts	10	4	4
5. App unavailable	1	1	1
6. App redirecting to a maintenance page	4	1	1
7. Heap memory issue with a node	1	1	1
8. App performance degradation	6	67	189
9. App failing with 503 error	1	1	1
10. Health check failing on 2 nodes	2	2	23
11. App redirecting to a maintenance page	6	10	22
12. Slowness in loading data	20	101	21
13. Performance alert about a node exceeding thresholds	0	0	0
Average false positives	4.9	23.2	32.3

Table 1: Number of false positives (FP) produced by each scheme for each incident, according to operator decided resolution (see § 6.2), for incidents in the dataset. Only FPs are shown, rather than false negatives, because the schemes were calibrated to have similar false negatives (§ 6.2).

Microbenchmark Results

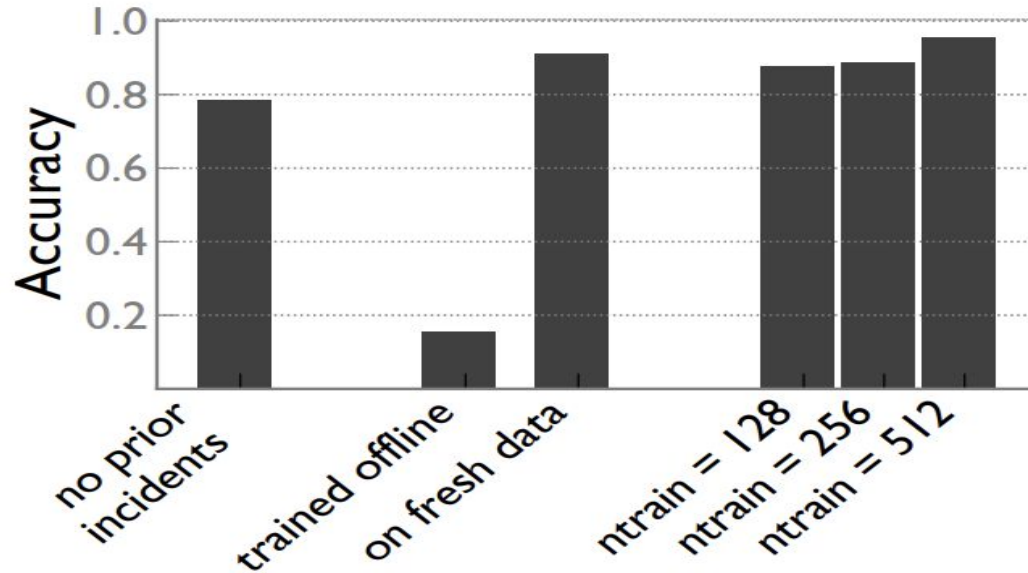
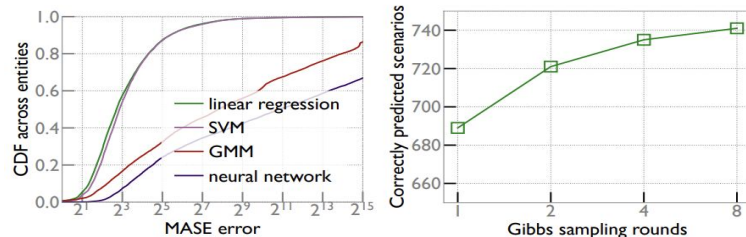


Figure 7: Various microbenchmarks with Murphy. Fresh data implies that Murphy was trained with data that included several minutes during which the incident was in progress.

Ablations and Sensitivity Analysis

Scheme	Missing values	Missing edge	Missing entity	Missing metric	Aggregate (avg(1-4))	Unchanged input
Murphy	0.84	0.75	0.78	0.81	0.80	0.86
Sage	0.64	0.67	0.66	0.82	0.70	0.80
NetMedic	0.16	0.20	0.16	0.22	0.18	0.22
ExplainIT	0.05	0	0	0	0.01	0

Table 2: Robustness: Accuracy with degraded/incomplete data. Numbers show recall in top-5. ExplainIT was designed for a different use case – as an interactive tool for queries posed by the user, hence its accuracy was low for automated diagnosis. Both Sage and Murphy were fairly robust.



(a) Error in metric prediction for 17K entities **(b) Multiple iterations of Gibbs sampling improves accuracy**

Figure 8: (a) Errors in predicting the metrics of an entity, given the metrics of all its neighbors across 17K entities spanning 300 production apps. (b) Gibbs sampling improved accuracy of prediction across multiple hops, consistent with the existence of cyclic effects in the production environment.

Critiques

- Production study is still limited, since it evaluates the method on only 13 real incidents
- Production ground truth is imperfect, because operator resolution does not always equal the true root cause
- Production calibration is narrow, since it is based on only 2 incidents with high-confidence ground truth
- Explanation stage is heuristic, because it uses thresholded labels and a simple causal state machine
- Method depends heavily on fresh online data, and performance drops when recent incident data is unavailable.

Future Directions

- Test method on a larger and more diverse set of real production incidents
- Look into richer nonlinear models beyond ridge regression for local probability modeling
- Try to upgrade explanation stage with learned or validated causal explanations instead of simple heuristic rules
- Look into how to make method less dependent on fresh online incident data

Implications

- Accurate root-cause diagnoses can help operators debug incidents faster and reduce time spent looking into how misleading symptoms come about
- A method that works with common monitoring telemetry is more practical for real enterprise systems than approaches that require fine-grained tracing
- Supporting cyclic and uncertain dependencies makes a method more realistic for modern cloud environments

Conclusion

- Presents a practical method for root-cause diagnosis in distributed cloud applications
- Main strength is that it works with common monitoring data and cyclic dependency structure
- Results show strong promise, especially in reducing false positives compared with prior methods
- Overall, the paper offers a useful step toward more practical and interpretable diagnosis in real cloud systems